Phase 5: Baselines and evaluation campaign

Fault tolerant control and post fault trajectory planning of a redundant cable driven parallel robot

M. J. Tachia, A. V. Maloletov

Technical report, 29 October 2026

About this document. This document records the evaluation of Phase 5: how the learned classifier enters the controller, the classifier retrained on both reference paths (Phase 4b), the settings of the threshold baseline B2, the protocol of the campaign with its one amendment, the checks made before the campaign was launched and, once it has run, its results. The protocol (`configs/evaluation_phase5.yaml`) was written before any run with an evaluation seed was simulated. Evaluation runs use seeds from 200000; the settings of every method were fixed on calibration seeds (100000 to 199999) or on the learning runs (from 300000), so no evaluation run shaped any method; evaluation runs prompted one change, amendment 1, whose settings were again chosen on calibration runs (Section 6). The campaign ran to completion; its results are in Section 9, and the probability aware tightening, which the specification put at the centre of the study, shows no measurable effect there.

1 What this phase does

The specification (Phase 0) asks for an evaluation in which the proposed controller and its baselines run on the same randomised runs, so that every comparison is paired, with the endpoints and the statistics fixed before the runs. Three things had to be settled before the campaign could be written down: how the learned classifier of Phase 4 takes part in closed loop (Section 2); on which data it is trained, since the campaign runs both reference paths and the Phase 4 model has seen only one (Section 3); and how the threshold baseline B2 is tuned, so that it is a fair opponent (Section 4). The protocol of Section 5 then fixes the methods, the runs, the metrics and the analysis. Table 1 lists the steps; each is a script of `scripts/`, and each output is stored in `results/`.

Table 1: Steps of Phase 5.

Step	Script	Uses
Helix runs for the classifier (training, validation)	`phase4b_generate.py`, `phase4b_dataset.py`	frozen controller, seeds 340000 to 350031
Five models on both paths	`phase4b_train.py`	Phase 4 hyperparameters
Temperature and thresholds per model, deployed model	`phase4b_calibrate.py`, `phase4b_freeze.py`	validation runs of both paths
Single offline test	`phase4b_test.py`	stored Phase 4 test runs
Gains of B2 before a fault	`phase5_b2_gains.py`	fault free calibration runs, seeds from 140000
Calibration runs of B2, thresholds	`phase5_b2_runs.py`, `phase5_b2_calibrate.py`	B2's own controller, seeds from 150000
Gains of B2 after a break declaration	`phase5_b2_postbreak.py`	B2's calibration break runs
Freeze of everything the campaign depends on	`phase5_freeze.py`	the files above
Campaign	`phase5_run.py`	evaluation seeds from 200000
Condensed rows, recovery times, statistics	`phase5_condense.py`, `phase5_analyse.py`	rows and traces of the campaign

2 How the learned classifier enters the controller

The controller takes three things from the IMM: its mode probabilities, which decide the declaration and, in the proposed controller P, weight the fault scenarios of the MPC before a declaration; the state of the platform, from its filters; and, after a declaration, the state and the tension deficit of the declared mode, which the accommodation of actuator faults and the planner use. A classifier gives only the first. In the learned baseline the classifier therefore replaces the mode probabilities, and the filters of the IMM keep running as the state estimator (`cdpr_ft/estimation/alternatives.py`, `cdpr_ft/control/ftc.py`). The learned baseline then differs from the IMM based controller in the diagnosis alone.

Two arms were built. In B3 the classifier declares and the MPC switches to the model of the declared mode after the declaration, as in the ablation A1, which keeps the IMM but switches off the probability aware tightening. This is how learned detectors are used in the literature on cable robots [Bettega et al., 2025]: detect, then reconfigure. In B3T the calibrated probabilities of the classifier also weight the fault scenarios

of the MPC before a declaration, as the IMM's do in P. Table 2 shows how the four controllers separate the effect of the diagnosis from that of the tightening.

Table 2: The four controllers that differ only in the diagnosis and in the use of the probabilities before a declaration.

Diagnosis	Tightening before the declaration	MPC switches after the declaration only
IMM (model based)	P	A1
Classifier (learned)	B3T	B3

B3, and not B3T, is the primary learned baseline because of what the first pilot run of B3T showed (pilot seed 900000, break of cable 1 on T1, with the Phase 4 model). As soon as the classifier was armed it gave the fault modes about 15% together, 4% the largest; the MPC tightened against the two most probable; the motion this produced was unlike anything in the classifier's training runs, which were run without tightening; within 30 ms its healthy probability fell below 10^{-3}, and an actuator fault of winch 2 was declared falsely 0.35 s after arming. The same classifier declaring only kept the healthy mode at 0.83 to 0.87 over the same interval and declared the break correctly. The IMM is less exposed to such a loop, because it takes the controller's own tension commands as its input and so has a model of the motion the controller causes; a classifier trained on runs without tightening has none. It is not immune either: in Phase 3 one calibration run showed the tightening amplify a passing doubt of the IMM from 0.36 to 0.98. The comparisons B3T against B3 and P against A1 measure this over the whole campaign.

Before a classifier's softmax outputs can weight scenarios or be compared with the thresholds of the declaration rule, they must behave as probabilities. One temperature per model is fitted on the validation windows by minimising the negative log likelihood of the true mode [Guo et al., 2017], and the thresholds are chosen on the calibrated outputs (Section 3).

3 The classifier trained on both paths (Phase 4b)

The Phase 4 classifier was trained on the circle T1 only, as the specification asks, so that the helix T2 would measure how a learned detector generalises to a path it has never seen. On T2 it raised a false alarm in 31 of 88 test runs. The campaign runs both paths, so a B3 that has never seen T2 would lose there for a reason its training data fix in advance. The baseline of the campaign is therefore a second model, trained on both paths by a protocol written before its runs (`configs/learning_phase4b.yaml`); the Phase 4 model stays in the paper as the result on generalisation.

Data and training. The new runs are 352 runs on T2 with the frozen controller in the configuration of the Phase 4 runs (variant `no_hedging`, declarations disabled, seeds 340000 to 350031): for every fault and cable, repetitions 0 to 5 go to training and 6 and 7 to validation, with 24 and 8 fault free runs. They are added to the Phase 4 training and validation runs of T1, so that both splits hold both paths in equal parts (528 and 176 runs); the test runs are the stored Phase 4 test runs of both paths. The network and its hyperparameters are those chosen by the Phase 4 search, without a new search, so that the two models differ in their training data only. Five seeds were trained; their best validation macro-F1 lies between 0.957 and 0.973 (Phase 4: 0.951 to 0.963).

Calibration, thresholds and the deployed model. The fitted temperatures lie between 0.55 and 0.64: the networks were under-confident, and calibration lowered the expected calibration error on the validation windows from 0.06 to 0.15 to between 0.002 and 0.005 (Table 3). On the calibrated outputs the thresholds of the declaration rule were chosen on the validation runs of both paths by the rule used for the IMM in Phase 3 and for the Phase 4 model: fewest runs with a false alarm, then most faults declared with the correct mode, then the smallest median delay of the break declarations, then the more conservative setting. In Phase 4 the validation macro-F1 had not ranked the seeds the way their declarations did, so the deployed seed is now chosen by the same key on the validation runs, the macro-F1 breaking ties. Seeds 2, 3 and 4 raise no false alarm on the validation runs; seed 2 declares most faults correctly and is deployed, with $\mu_{\text{iso}} = 0.9999$, $N_c = 4$ and $T = 0.638$ (`configs/classifier_phase4b.yaml`).

Single offline test. The deployed model was evaluated once on the stored Phase 4 test runs (88 per path, 80 with a fault), with the IMM (frozen thresholds 0.9999 and 2) and the Phase 4 model on the same runs (Table 4). On T1 it raises no false alarm and declares 77 of the 80 faults correctly; on T2 it declares 78 correctly, and three of the eight fault free runs carry a false alarm, against 31 of 88 runs for the Phase 4 model.

Table 3: The five models of Phase 4b on the validation runs of both paths (176 runs, 160 with a fault). NLL: mean negative log likelihood of the true mode over the validation windows; ECE: expected calibration error, before and after temperature scaling. Delays: medians of the correct declarations.

Seed	Macro-F1	T	NLL	ECE	μ_{iso}, N_c	False alarm runs	Correct of 160	Delay break / LoE / bias (ms)
0	0.973	0.598	0.167 → 0.090	0.093 → 0.004	0.9999, 4	3	159	28 / 31 / 54
1	0.966	0.599	0.172 → 0.098	0.093 → 0.005	0.9999, 8	1	158	48 / 51 / 66.5
2	0.965	0.638	0.153 → 0.092	0.078 → 0.002	0.9999, 4	0	157	29 / 32 / 74
3	0.966	0.643	0.125 → 0.077	0.062 → 0.002	0.9999, 6	0	154	39.5 / 44 / 112
4	0.957	0.550	0.258 → 0.116	0.153 → 0.003	0.9999, 6	0	155	42 / 52 / 133

Against the Phase 4 model on T2 the paired 95% bootstrap intervals are +31 to +52 percentage points for correct declarations and −42 to −23 for runs with a false alarm. Training on both paths has removed most of the failure to generalise. Two differences to the IMM remain: the classifier still declares later, by 10 to 15 ms for breaks and 8 to 13 ms for losses of effectiveness (paired intervals of the median differences, both paths), and its false alarms on the helix are not gone. Over the five seeds the classifier declares 96.7 ± 0.7% of the T1 faults and 97.8 ± 1.0% of the T2 faults correctly (mean ± standard deviation), with no false alarm on T1 and 2.2 ± 1.1 runs with a false alarm on T2. The campaign measures these false alarms on 100 fault free runs per path.

Table 4: Single offline test of the Phase 4b model on the stored Phase 4 test runs. Delays: median of the correct declarations, ms.

	T1			T2		
	Phase 4b	Phase 4	IMM	Phase 4b	Phase 4	IMM
Runs with a false alarm, of 88	0	1	0	3	31	0
Faults declared correctly, of 80	77	73	80	78	45	80
Missed / declared as another mode	0 / 3	1 / 5	0 / 0	0 / 2	0 / 10	0 / 0
Breaks, LoE, biases correct (of 16, 32, 32)	16, 32, 29	16, 31, 26	16, 32, 32	16, 32, 30	8, 22, 15	16, 32, 32
Delay break / LoE / bias	29 / 32 / 75	31 / 38 / 106	15 / 23 / 43	28 / 32 / 64	32 / 41.5 / 93	17 / 22 / 39.5
Window accuracy / macro-F1	0.988 / 0.966	0.980 / 0.933		0.977 / 0.954	0.591 / 0.700	

4 The threshold baseline B2

B2 stands for the classical pipeline: residual thresholds on the winch quantities for detection and isolation, a computed torque law with tension distribution (the max-margin linear programme of the methods) instead of the MPC, and the same planner library as P after a break. A baseline is only informative if it is tuned with the same care as the method it is compared with, so every setting of B2 was chosen on calibration runs of its own controller, by rules stated before the runs.

Gains before a fault. The gains of the computed torque law had been set in Phase 3 without a study. Settings with critical damping, natural frequency ω ($k_p = \omega^2$, $k_d = 2\omega$), were compared on eight fault free calibration runs (seeds from 140000, both paths, declarations disabled). The first version of the study compared 10, 20 and 30 rad/s and took the smallest median error among the settings that lose no run. It chose 20 rad/s, although its runs left the tension bounds for up to 32.7% of the time from 2 s on, against none at 10 rad/s for a median error larger by 0.5 mm, and 30 rad/s lost runs: the chosen loop was near the edge of stability. The first eight B2 runs of the campaign showed the consequence (Section 6), and the rule was replaced by one that requires the bounds to be kept: among the settings that lose no run and keep every cable within its bounds for at least 99% of the time in every run, the smallest median error. On a grid refined below and between the first settings (Table 5) it chooses 15 rad/s, with a median RMS error of 4.7 mm and no time outside the bounds. Settings 10, 20 and 30 rad/s were run again and reproduced the first study run by run.

Thresholds. B2 declares a break of cable i when the winch estimate of its tension lies below θ_{b}, and an actuator fault of winch i when the magnitude of its tension residual exceeds θ_{a}, in N_c consecutive estimator cycles. Its winch quantities are not distributed as in the runs of the MPC, so the thresholds were chosen on 176 calibration runs of B2's own controller with the chosen gains (seeds from 150000: the five faults of the learning runs on every cable and both paths, two repetitions, eight fault free runs per path, declarations disabled), by the lexicographic rule used for the IMM and for B3. Every actuator threshold of the grid that detected an actuator fault also raised false alarms with this controller, so the rule sets $\theta_{\text{a}} = \infty$; with $\theta_{\text{b}} = 9$ N

Table 5: Gains of B2 before a fault, on eight fault free calibration runs (four per path). Error: RMS position error from 2 s; outside: share of the time with a cable outside $[t_{\min}, t_{\max}]$, from 2 s.

ω (rad/s)	k_p	k_d	Lost	Median error (mm)	Largest outside (%)	Smallest margin (N)
5	25	10	0	47.0	0	6.8
7.5	56.25	15	0	20.3	0	12.9
10	100	20	0	10.4	0	16.1
12.5	156.25	25	0	6.8	0	12.8
15	225	30	0	4.7	0	13.6
20	400	40	0	9.9	32.7	−42.2
30	900	60	2	24.2	80.6	−381.5

and $N_c = 1$ B2 declares all 32 breaks correctly (median delay 21 ms, largest 26 ms) with no false alarm. It does not isolate actuator faults, and in 7 of its 64 calibration runs with a loss of effectiveness ($\rho = 0.6$ in all seven) the faulty winch was an upper one and the tension of a lower cable fell below θ_{b} after the onset, so that a break of that cable was declared. B2 is thus a break detector, which is what the classical literature on cable robots offers.

Gains after a break declaration. The calibration runs have declarations disabled and never exercise what B2 does after a declaration. Run as a whole on the 32 break runs of its calibration set, continued 4 s after the onset as in the campaign, B2 declared every break but lost the platform in 24 of them: the computed torque law, following the recovery plan with the gains it used before the fault, drove the platform past the tilt it could hold on seven cables. The proposed controller lowers its MPC weights on velocity and raises that on tension rates after a break declaration, a setting fixed in Phase 3 on calibration runs. So that B2 is treated alike, its gains after a break declaration were chosen on these 32 runs by a rule stated before them: the fewest lost runs, then the smallest median share of the transient window outside the bounds, then the smallest median error after the transient (Table 6). It chooses 7.5 rad/s, which loses 8 of the 32 runs (6 after an upper and 2 after a lower cable). The option is a pair of gains used by the computed torque law after a break declaration (`ct_gains_post`); it is used by B2 only.

Table 6: Gains of B2 after a break declaration, on the 32 break runs of its calibration set (16 upper, 16 lower cables). All breaks were declared correctly.

ω after the declaration (rad/s)	Lost	Upper / lower	Median outside, transient (%)	Median error after it (mm)
5	20	12 / 8	52.0	45.0
7.5	8	6 / 2	25.5	27.1
10	10	4 / 6	44.1	27.7
12.5	17	6 / 11	70.5	39.5
15 (unchanged)	24	9 / 15	72.5	79.8

5 Protocol of the campaign

Methods. Table 7 lists the nine methods. All run on the frozen controller of Phase 3 (`configs/controller_phase3.yaml`) and differ only where the table says so.

Table 7: Methods of the campaign.

Method	What it is	Runs
P	Proposed: IMM, probability aware MPC, recovery plans from the library	all
A1	No tightening before the declaration; the MPC switches after it	all
A2	No re-planning after a break declaration	breaks
A5	Planner solved online, with its latency, instead of the library	breaks
B0	No fault tolerance: MPC with the healthy model throughout	fault runs
B1	Oracle: the true fault declared at its onset	fault runs
B2	Threshold detection, computed torque with tension distribution, library plans	all
B3	Classifier (Phase 4b) in place of the IMM probabilities for the declaration	all
B3T	As B3, the classifier's probabilities also tighten the MPC	repetitions 0 to 3; 30 fault free runs per path

Runs. Both reference paths, T1 (circle) and T2 (helix); the three fault types, a break, a loss of effectiveness ($\rho = 0.3$ and 0.6, alternating with the repetition) and a torque bias ($\pm 10\%$ of the holding torque, alternating), on every cable; 13 repetitions, so 104 runs per path and fault type and 624 fault runs, with the sensor noise nominal or doubled every two repetitions and the onset drawn uniformly in $[3, 7]$ s. A fault run ends 4 s after its onset, or when the platform is lost. There are 100 fault free runs of 16 s per path. The plant parameters are randomised from the seed as in the specification. Every method runs on the same seeds, so every comparison is paired. B0, B1, A2 and A5 skip the runs on which they would repeat another method exactly, and B3T, about twice as costly per run, runs on a subset fixed in advance: 5,212 jobs in all.

Metrics. Every run writes one row of metrics and a trace at 200 Hz (`cdpr_ft/eval/campaign.py`). Errors are taken to the active reference, the nominal path before a declaration and the controller's reference (plan, hold pose) after it, in three windows: before the fault (from 2 s, the arming time), the transient (2 s from the onset) and after it. In each window the RMS and largest position and rotation errors, the smallest margin of the intact cables to their bounds and the share of the window with a cable outside them. The detection metrics are the declared mode and its delay, false alarms (a declaration before the onset or in a fault free run, counted per hour of armed time) and missed faults. The outcome is whether the platform is lost (tilt above 30°, frame or drum window left), and for breaks the planner branch and the distance to the target of the plan. The recovery time is the time from the onset after which the position error stays within a band for 1 s, the band being twice the 95th percentile of the error in the fault free runs of the same path. The deficit error is the RMS error of the IMM's tension deficit against the true deficit of the plant after an actuator declaration. Computation times of the MPC, the IMM and the classifier are logged per cycle.

Analysis. Six primary endpoints were fixed, each with its comparisons and a Holm correction within it [Holm, 1979]: the platform lost after a break (P against B0, B2, B3 and A2); the fault declared with the correct mode (P against B2 and B3, A1 against B3); the delay of correct declarations (the same comparisons); false alarms per hour of armed fault free operation (P against A1, B2 and B3); the smallest tension margin in the transient (P against A1, B3T against B3); and the share of the transient outside the bounds (P against A1 and B1). Shares carry 95% Wilson intervals [Wilson, 1927] and paired differences a bootstrap interval with McNemar's exact test [McNemar, 1947]; continuous endpoints a median with a bootstrap interval and the Wilcoxon signed rank test for paired differences [Wilcoxon, 1945]; false alarm rates an exact Poisson interval. Bootstraps use 2000 resamples over runs [Efron and Tibshirani, 1993]. Every run is reported and none excluded. The secondary list covers the tracking errors, recovery times, planner branches, the deficit error, the online planner, the 2×2 table of Table 2, and every result by path, fault type, cable group and noise level.

Not run. Double faults, because the IMM and the declaration logic handle one fault and do not re-arm after a declaration; the ablations A3 (variants of the mode set) and A4 (an external pose sensor), which were not built; and point to point paths (T3), which were not designed in Phase 1. Each is a limitation or further work in the paper.

6 Amendment to the protocol

The campaign was first started at 14:20 UTC on 27 October 2026 with B2 at the gains of the first version of its gain study (20 rad/s). After 75 rows, all on the breaks of repetition 0 on T1, the B2 rows showed tensions outside their bounds for 31 to 45% of the window before the fault and the platform lost in all eight runs, with the loss of one upper cable run 0.2 s after the onset, before any declaration. The gain study's own runs had shown the same tendency at 20 rad/s, which its rule did not weigh. The campaign was stopped; B2's gains were chosen again on the same calibration runs with a rule that keeps the bounds, its thresholds calibrated again on its own runs with these gains, and its gains after a break declaration chosen on its calibration break runs (Section 4). No setting of any other method changed, and the code change is an option used by B2 alone; the quick check of the proposed controller gave the stored values exactly after it. The protocol records the change as amendment 1. The campaign was then started again from its first run at 17:53 UTC under the amended protocol; the 75 rows of the first start are kept in `results/phase5_first_start/` as a record and enter no result.

This change strengthens a baseline against which the proposed controller is compared, and it was made on calibration runs, but it was prompted by evaluation runs. The manuscript states this.

7 Checks before the launch

- *Tests.* The 56 automated tests of the repository pass, five of them new in this phase: the vectorised threshold rule used to choose B2's thresholds equals the rule the controller runs cycle by cycle; the classifier fed sample by sample in closed loop gives the probabilities the offline evaluation computes from the same windows; the run list is paired and in the evaluation range; the variant groups of the controller are consistent; and the B2 settings of the configuration file reach the controller, the post-break gains for B2 only.
- *Proposed controller unchanged.* The baselines added code to the controller. The quick check (the frozen controller on the eight calibration breaks, compared value by value with the stored final check of Phase 3) gave all 432 values identical, after the first change, after the last change before the first start and again after the code change of the amendment. Two Phase 4 runs simulated again with the code of the first start were identical to the stored records (502 191 values).
- *Pilot.* 56 jobs on pilot seeds (from 900000) timed the methods and checked the rows and traces; no setting was chosen on them. They also revealed the feedback loop of B3T (Section 2) and the bias of the accommodation (Section 8).
- *Freeze.* `results/phase5_freeze.json` holds the hashes of the protocol (`10331039ba40774c`), the controller configuration (`71c8e8640f29365f`), the B3 configuration and model (`813b05ed8607c087`), the B2 configuration (`aee6e9a39eb377fa`), the recovery library and safe poses, and the digests of the controller code and its configuration files (`2e03873280e4d102`) and of the learning code (`8aa62569041d0e4b`). The runner, and every worker process when it starts, refuse to run if any of them has changed.

8 Accommodation of actuator faults: a finding

After an actuator declaration the frozen controller corrects the tension loop of the faulty winch by the deficit that the IMM's filter of the declared mode estimates, while the IMM keeps the tension commands as its input. That filter then estimates the deficit left after the correction, and the pair settles at about half of the true deficit: in the pilot, 27 to 32 N against 54 to 63 N for $\rho = 0.3$, with a position error after the fault of about 25 mm. Giving the IMM the corrected command removes this bias in the steady state; on four pilot runs it cut the error after the fault from 25 to 2 mm for $\rho = 0.3$, but it lost the platform for $\rho = 0.6$ on both paths, where the frozen scheme kept it. The frozen scheme was kept and its code left unchanged; the campaign reports the deficit error of every actuator run against the true deficit of the plant. In the paper this is a limitation with a clear cause and a direction for further work: an accommodation that estimates the deficit from the corrected command needs a stability argument the present design does not have.

9 Results

The campaign. All 5,212 jobs of the amended protocol ran between 17:53 UTC on 27 October and 01:58 UTC on 29 October 2026, two at a time on the two cores of the machine (32 h of wall time, 22 s per job on average including the restarts). The container of the machine was restarted twice, at 23:57 UTC on 27 October and at 23:50 UTC on 28 October; the runner was started again each time and skipped the jobs that already had a row, and the jobs that were running at the restart were run again from their seeds. No row is missing and none was excluded. The 67 rows of the first start that belong to methods other than B2 came out identical, value for value, when the campaign ran them again. Every number below is read from `results/phase5/analysis.json`, and every table of this section is written from that file by `scripts/phase5_tables.py`.

Primary endpoints. Table 8 gives the six endpoints with their comparisons; every comparison that differs does so after the Holm correction within its endpoint.

- *Platform lost after a break.* The proposed controller lost the platform in 11 of the 208 break runs (5.3%), against 122 for B0 (58.7%), 65 for B2 (31.2%), 25 for B3 (12.0%) and 38 for A2 (18.3%). All 11 losses of P followed the break of an upper cable (6 of 52 on T1, 5 of 52 on T2); it kept the platform after every lower cable break. The oracle B1, which knows the fault at its onset, lost 4.
- *Correct declaration.* P declared 623 of the 624 faults with the correct mode and A1 all 624; B3 declared 598 correctly, and B2 208, which are the 208 breaks, since it isolates no actuator fault.

Table 8: Primary endpoints. Shares: count of pairs with the event and 95% Wilson interval (%); false alarms: count in the armed hours, exact Poisson interval per hour; differences: first method minus second, 95% bootstrap interval (mean for shares, median for continuous endpoints). p: McNemar's exact test, Wilcoxon signed rank test or conditional binomial test for rates; Holm: adjusted within the endpoint. Continuous endpoints are compared run by run, so only the paired difference is given.

Endpoint, comparison	Pairs	First	Second	Difference	p	Holm
Platform lost after a break						
P vs B0	208	11/208 [3, 9]	122/208 [52, 65]	[−60.1, −46.2] pp	<0.001	<0.001
P vs B2	208	11/208 [3, 9]	65/208 [25, 38]	[−32.7, −19.2] pp	<0.001	<0.001
P vs B3	208	11/208 [3, 9]	25/208 [8, 17]	[−11.5, −2.4] pp	0.007	0.007
P vs A2	208	11/208 [3, 9]	38/208 [14, 24]	[−18.8, −7.7] pp	<0.001	<0.001
Fault declared with the correct mode						
P vs B2	624	623/624 [99, 100]	208/624 [30, 37]	[62.7, 70.0] pp	<0.001	<0.001
P vs B3	624	623/624 [99, 100]	598/624 [94, 97]	[2.4, 5.6] pp	<0.001	<0.001
A1 vs B3	624	624/624 [99, 100]	598/624 [94, 97]	[2.7, 5.8] pp	<0.001	<0.001
Declaration delay of correct declarations						
P vs B2	208	–	–	−5.0 [−5.0, −5.0] ms	<0.001	<0.001
P vs B3	597	–	–	−10.0 [−15.0, −10.0] ms	<0.001	<0.001
A1 vs B3	598	–	–	−10.0 [−15.0, −10.0] ms	<0.001	<0.001
False alarms per hour of armed fault-free operation						
P vs A1	824	1 in 1.29 h [0.0, 4.3]	0 in 1.29 h [0.0, 2.9]	–	1.000	1.000
P vs B2	824	1 in 1.29 h [0.0, 4.3]	4 in 1.29 h [0.8, 7.9]	–	0.219	0.437
P vs B3	824	1 in 1.29 h [0.0, 4.3]	42 in 1.29 h [23.4, 43.9]	–	<0.001	<0.001
Smallest tension margin in the transient window						
P vs A1	624	–	–	0.0 [0.0, 0.0] N	0.262	0.262
B3T vs B3	138	–	–	−19.0 [−28.7, −13.8] N	<0.001	<0.001
Share of the transient window with a tension outside its bounds						
P vs A1	624	–	–	0.0 [0.0, 0.0] pp	0.496	0.496
P vs B1	624	–	–	0.0 [0.0, 0.0] pp	<0.001	<0.001

- *Delay.* On the same runs P declared 5 ms earlier than B2 and 10 ms earlier than B3 (paired medians). Its break declarations took 17 ms (median; at most 19 ms), those of B2 22 ms and those of B3 30 ms.
- *False alarms.* P raised one false alarm in 1.29 h of armed operation before any fault (0.8 per hour, 95% interval 0.02 to 4.3), A1 none, B2 four, and B3 42 (32.5 per hour, 23.4 to 43.9); the difference to B3 is significant, that to B2 is not. The false alarm of P came 2.8 s before a loss of effectiveness of the same winch, on a run with doubled noise.
- *Tension margin in the transient.* P and A1 do not differ (paired median difference 0 N, $p = 0.26$). With the classifier, the tightening lowers the margin by 19 N (B3T against B3, interval −28.7 to −13.8 N).
- *Time outside the bounds in the transient.* P and A1 do not differ. Against the oracle, P spends more of the transient with a cable outside its bounds: after breaks 5.1% against 2.1% (medians), which is what the 17 ms of detection cost.

After a break. Table 9 shows the outcome by fault type. Without fault tolerance (B0) an upper cable break loses the platform in every run (104 of 104) and a lower one in 18 of 104. With P the error returned to the band of the fault-free runs within 1 s in 181 of the 208 break runs, with a median recovery time of 0.27 s (oracle 0.19 s). Re-planning carries much of this: without it (A2) 38 runs were lost, 86.5% of the transient was spent outside the bounds (median) and the error after the transient was 72.7 mm against 5.6 mm. The plan library matters as well: solving the plan online, with its latency (A5), lost 20 runs against 11 (paired, $p = 0.09$, a secondary comparison) and raised the time outside the bounds from 5.1 to 20.2%. The classical pipeline B2, with gains tuned on its own calibration runs and the same plans, lost 65 runs, 30 of them after a lower cable break.

After an actuator fault. Among the methods that declare, only B2 and B3T lost the platform after a loss of effectiveness or a bias: B2 in 19 and 2 runs, as it cannot isolate actuator faults and in 21 loss of effectiveness runs declared a break of a healthy cable, and B3T in 29 and 22 of its 64 runs, after false alarms. B0, which does not react, lost one. The learned classifier isolated 206 of the 208 losses of effectiveness but only 184 of the 208 biases, 21 of which it took for a fault of another winch. The accommodation of the IMM leaves the bias found in the pilot (Section 8): after a loss of effectiveness the deficit is estimated with an RMS

Table 9: Fault runs by method and fault type, both paths. Lost: platform lost; correct: fault declared with the correct mode; delay: median of the correct declarations (ms); margin: median of the smallest tension margin in the transient (N); outside: median share of the transient with a cable outside its bounds (%); error: median RMS position error after the transient (mm); recovered: error back in the band of the fault-free runs for 1 s before the run ended, and the median time to that (s).

Fault	Method	Runs	Lost	Correct	Delay	Margin	Outside	Error	Recovered	Time
Break	P	208	11/208	208/208	17	−10.0	5.1	5.6	181/208	0.27
	A1	208	11/208	208/208	17	−10.0	5.2	5.7	182/208	0.28
	A2	208	38/208	208/208	17	−24.5	86.5	72.7	74/208	1.01
	A5	208	20/208	208/208	17	−10.2	20.2	5.5	173/208	0.70
	B0	208	122/208	0/208	–	−31.9	79.4	83.4	0/208	–
	B1	208	4/208	208/208	0	−8.2	2.1	4.8	197/208	0.19
	B2	208	65/208	208/208	22	−38.7	19.5	23.8	93/208	0.90
	B3	208	25/208	208/208	30	−10.0	8.8	6.3	157/208	0.33
	B3T	64	44/64	11/64	532	−76.7	81.8	70.2	19/64	0.43
Loss of effectiveness	P	208	0/208	207/208	23	26.9	0.0	18.2	80/208	0.00
	A1	208	0/208	208/208	23	26.7	0.0	18.2	80/208	0.00
	B0	208	1/208	0/208	–	24.5	0.0	31.9	55/208	0.00
	B1	208	0/208	208/208	0	27.3	0.0	18.2	80/208	0.00
	B2	208	19/208	0/208	–	17.5	0.0	62.2	48/208	0.45
	B3	208	0/208	206/208	33	26.5	0.0	18.2	80/208	0.00
	B3T	64	29/64	5/64	32	4.0	0.0	33.7	35/64	0.00
Bias	P	208	0/208	208/208	41	37.0	0.0	5.3	206/208	0.00
	A1	208	0/208	208/208	42	37.1	0.0	5.3	206/208	0.00
	B0	208	0/208	0/208	–	36.0	0.0	8.1	153/208	0.00
	B1	208	0/208	208/208	0	37.2	0.0	5.4	206/208	0.00
	B2	208	2/208	0/208	–	35.3	0.0	16.1	199/208	0.00
	B3	208	0/208	184/208	79	37.0	0.0	5.7	200/208	0.00
	B3T	64	22/64	0/64	–	17.1	0.0	18.4	42/64	0.00

error of 30 N (median; 20 N for $\rho = 0.3$ and 52 N for $\rho = 0.6$), and the error after the transient is 18.2 mm (14 and 45 mm), so that only 80 of the 208 runs return to the band of the fault-free runs. After a bias the deficit error is 5.6 N and 206 of 208 runs return to the band.

Table 10: Fault-free runs by method and path. False alarms per hour of armed time (from 2 s) with an exact Poisson interval; error: median RMS position error (mm); margin: median of the smallest tension margin (N).

Method	Path	Runs	False alarm runs	Armed hours	Per hour	Error	Margin
P	T1	100	0	0.39	0.00 [0.00, 9.49]	3.04	36.2
	T2	100	0	0.39	0.00 [0.00, 9.49]	3.35	18.4
A1	T1	100	0	0.39	0.00 [0.00, 9.49]	3.04	36.2
	T2	100	0	0.39	0.00 [0.00, 9.49]	3.35	18.4
B2	T1	100	1	0.39	2.57 [0.07, 14.33]	7.63	35.1
	T2	100	0	0.39	0.00 [0.00, 9.49]	6.86	18.1
B3	T1	100	4	0.39	10.29 [2.80, 26.34]	3.04	36.2
	T2	100	34	0.39	87.43 [60.55, 122.17]	3.35	18.5
B3T	T1	30	22	0.09	253.18 [158.67, 383.32]	41.76	−127.4
	T2	30	26	0.08	307.09 [200.60, 449.96]	38.79	−83.1

Fault-free operation. On the 200 fault-free runs (Table 10) P and A1 raised no false alarm and tracked the paths with a median RMS error of 3.0 mm on T1 and 3.4 mm on T2; B2 tracked at 7.6 and 6.9 mm. The learned classifier raised false alarms in 4 of the 100 fault-free runs on T1 and in 34 of the 100 on T2. Offline, on the eight fault-free test runs of T2, it had raised three (Section 3); in closed loop the rate on T2 is about the same, 34%, now measured on a hundred runs. Training on both paths has thus brought the classifier's isolation close to that of the IMM, but not its rate of false alarms on the helix.

The probability aware tightening. The 2×2 comparison (Table 11) separates the diagnosis from the use of its probabilities before a declaration. With the IMM the tightening has no measurable effect: P and

Table 11: Diagnosis and tightening on the runs of B3T (192 fault runs, 60 fault-free runs, the same for the four methods).

Method	Lost after a break	Correct	Delay (ms)	False alarm runs	Per hour	Margin (N)	Outside (%)
P	4/64	192/192	23	0	0.00 [0.00, 9.47]	26.2	0.0
A1	5/64	192/192	23	0	0.00 [0.00, 9.47]	26.2	0.0
B3T	44/64	16/192	137	154	505.11 [428.48, 591.48]	−10.0	11.2
B3	8/64	186/192	34	11	28.25 [14.10, 50.55]	26.0	0.0

A1 lose the same number of runs after a break (11 each over the campaign, the outcome differing in ten runs, five each way), declare the same faults at the same delays, and have the same margins and time outside the bounds. Before a fault the tightening is almost never active: the tracking error before the fault is identical in 804 of the 824 paired runs. The reason is the speed of the IMM. Every break was declared within 19 ms, less than the 20 ms period of the MPC, so the MPC solves at most once with intermediate probabilities before it switches to the model of the declared mode. The tightening can only act while the diagnosis is uncertain, and with this diagnosis that interval is shorter than one control period. With the classifier, whose probabilities are not those of a model of the closed loop, the tightening is harmful, as the pilot suggested: B3T raised a false alarm in 154 of its 252 runs against 11 for B3 on the same runs, declared 16 of 192 faults correctly against 186, and lost 44 of 64 break runs against 8.

Table 12: Computation per cycle on the machine of the campaign (two runs in parallel on two cores), fault runs: medians over runs of the mean and the 99th percentile, and the largest value (ms). Periods: MPC 20 ms, estimator 5 ms. The estimator cycle includes the alternative diagnosis of B2, B3 and B3T, whose classifier alone is shown in the last column. B2 has no MPC.

Method	MPC mean	MPC p99	MPC max	Estimator mean	Estimator p99	Classifier mean
P	32.0	65.1	362.3	2.89	5.31	–
A1	32.0	64.2	407.5	2.88	5.30	–
A2	44.8	124.5	414.7	2.89	5.28	–
A5	34.4	89.3	349.4	2.91	5.34	–
B0	34.2	70.4	355.0	2.92	5.34	–
B1	32.3	63.8	277.5	2.88	5.29	–
B2	–	–	–	3.45	6.08	–
B3	31.9	66.0	323.4	4.65	7.56	1.48
B3T	90.3	637.2	1142.6	2.94	4.80	0.94

Computation. Table 12 gives the times on the machine of the campaign, with two runs sharing two cores. The estimator cycle (observers and IMM) takes 2.9 ms on average and 5.3 ms at the 99th percentile, against a period of 5 ms; with the classifier it takes 4.7 ms. The MPC takes 32 ms on average and 65 ms at the 99th percentile, against a period of 20 ms. The simulation waits for each solve, so the closed loop behaves as if the solver met its period; on hardware the MPC as implemented here (CasADi and IPOPT with the models compiled to C) would not, which is a limitation, with the options (a real-time iteration scheme, a faster solver, a shorter horizon).

What these results support. The results support three claims: that a model-based diagnosis (IMM over seventeen modes) isolates every fault type quickly and with almost no false alarm, faster and far more reliably than a learned classifier trained on the same information and than a threshold rule; that a library of recovery plans computed offline, used after a break declaration, keeps the platform after 95% of the breaks, against 41% without fault tolerance, 69% with the classical pipeline, 82% without re-planning and 90% with the planner solved online; and that a learned diagnosis must not be fed into the constraints of the controller, which then sees motion its training data never contained. They do not support the claim that the probability aware tightening of the MPC improves safety after a fault: with a diagnosis this fast it has nothing to act on.

10 Reproduction

The campaign wrote 1.7 GB of rows and traces to `results/phase5_work/`, which is not kept. `scripts/phase5_condense.py` keeps every row (`results/phase5/rows.jsonl.gz`, 1.3 MB), with the recovery time of

every fault run computed from its trace, and the 388 traces fixed by the protocol (repetition 0 of every path, fault type and cable, and fault-free repetitions 0 and 1, for every method that ran them; about 110 MB). Every other run can be simulated again from its seed.

`reproduce.py phase5 --only campaign_subset,analysis` simulates 28 jobs of the campaign again (every method on four runs fixed in `scripts/phase5_check.py`: a break of cable 1 on T1, a loss of effectiveness of winch 5 on T2, a bias of winch 8 on T1 and the first fault-free run of T2) and recomputes the statistics from the stored rows. Run on the machine of the campaign after it had finished, it gave the stored rows and traces identically (1 437 row values and 3 885 579 trace values, timings left out) in 7.4 min, and the analysis file identically (37 758 values). With the identical rerun of the first-start rows, this shows that the stored campaign is what the code of the repository gives on this platform. The B2 steps (`b2_gains`, `b2_runs`, `b2_thresholds`, `b2_postbreak`) take about 55 min; settings 10, 20 and 30 rad/s of the gain study already reproduced the first study run by run.

References

J. Bettega, G. Piva, D. Richiedei, and A. Trevisani. Load torque estimation for cable failure detection in cable-driven parallel robots: a machine learning approach. *Multibody System Dynamics*, 64:485–513, 2025. doi: 10.1007/s11044-024-10023-3.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proc. 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330, 2017.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6 (2):65–70, 1979.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.